

TP Introducción a Bases de Datos

GRUPO 11

Catania - Degrossi - Rodríguez

Implementación del Modelo Relacional

En esta primera etapa diseñamos e implementamos, sobre PostgreSQL, el modelo relacional de una cadena de fábricas de pastas formada por franquicias independientes de la Ciudad de Buenos Aires. El objetivo fue construir una base de datos sólida, coherente y bien poblada que sirviera de cimiento para los análisis de las etapas siguientes.

1.1 Elección del Modelo

Elegimos modelar una cadena de fábricas de pastas integrada por franquicias independientes de CABA. Cada franquicia maneja su propio catálogo de pastas, sus clientes y sus ventas. Nos pareció un dominio interesante porque combina varios mundos (catálogo de productos, recetas, clientes y operaciones de venta) y, sobre todo, porque genera volumen y variedad de datos suficientes para que las consultas de las etapas posteriores sean ricas y representativas.

A la hora de diseñar el esquema tomamos una serie de decisiones pensando en reflejar fielmente el dominio sin agregar complejidad innecesaria. La geografía se modela en tres niveles encadenados, barrio, calle y dirección: como todo ocurre dentro de CABA, el barrio es el nivel raíz y es quien guarda el código postal, por lo que no hizo falta modelar país ni provincia. Cada franquicia se identifica por su “sello”, una clave natural del estilo “FR-001”. Las pastas usan una clave compuesta por el sello de la franquicia y el código de pasta; de esta forma una misma pasta pertenece a una franquicia concreta y cada local puede fijar su propio precio. Además, la pasta se especializa en dos subtipos, pasta seca y pasta rellena, en una jerarquía disjunta y total (toda pasta es de uno de los dos tipos, y nunca de ambos a la vez).

Los rellenos se arman a partir de ingredientes mediante una relación de muchos a muchos (la tabla `relleno_ingredientes`), que además guarda la cantidad de cada ingrediente. Cada cliente pertenece a una franquicia y, opcionalmente, puede tener una pasta favorita, que siempre debe ser de esa misma franquicia. Finalmente, las ventas se representan con una compra que se descompone en líneas de detalle (qué pasta y cuántos kilos), y cada línea conserva el precio al momento de la venta, de modo que cambios de precio posteriores no alteran el historial. Por estar fuera del alcance pedido, deliberadamente no modelamos stock, empleados ni proveedores; y, como el enunciado lo prohíbe en esta etapa, tampoco creamos índices.

Para que la base no admita datos incoherentes, acompañamos el modelo con un conjunto de reglas de negocio. Todos los montos y cantidades deben ser positivos (precio por kilo, precio unitario, cantidad de kilos, cantidad de ingredientes y número de puerta), y el

promedio de kilos diarios no puede ser negativo. El tipo de pasta sólo puede ser “S” (seca) o “R” (rellena). Las fechas tienen que ser coherentes: una franquicia no puede iniciar en el futuro, no puede haber compras futuras y la fecha de nacimiento de un cliente tampoco puede ser posterior a hoy.

También cuidamos la unicidad donde corresponde: el nombre de cada barrio es único, una calle es única dentro de su barrio, el email de cada franquicia es único y el documento de un cliente es único dentro de su franquicia. Cada dirección pertenece a un único dueño (o una franquicia o un cliente, nunca a ambos) y, como ya mencionamos, la pasta favorita de un cliente debe ser de su propia franquicia. También tenemos que no es obligatorio que todos los clientes tengan pasta favorita y no es necesario que para registrar una pasta esta deba tener una venta. Por último, dos reglas se garantizan con triggers: un relleno no puede tener más de seis ingredientes y una franquicia no puede superar los mil clientes.

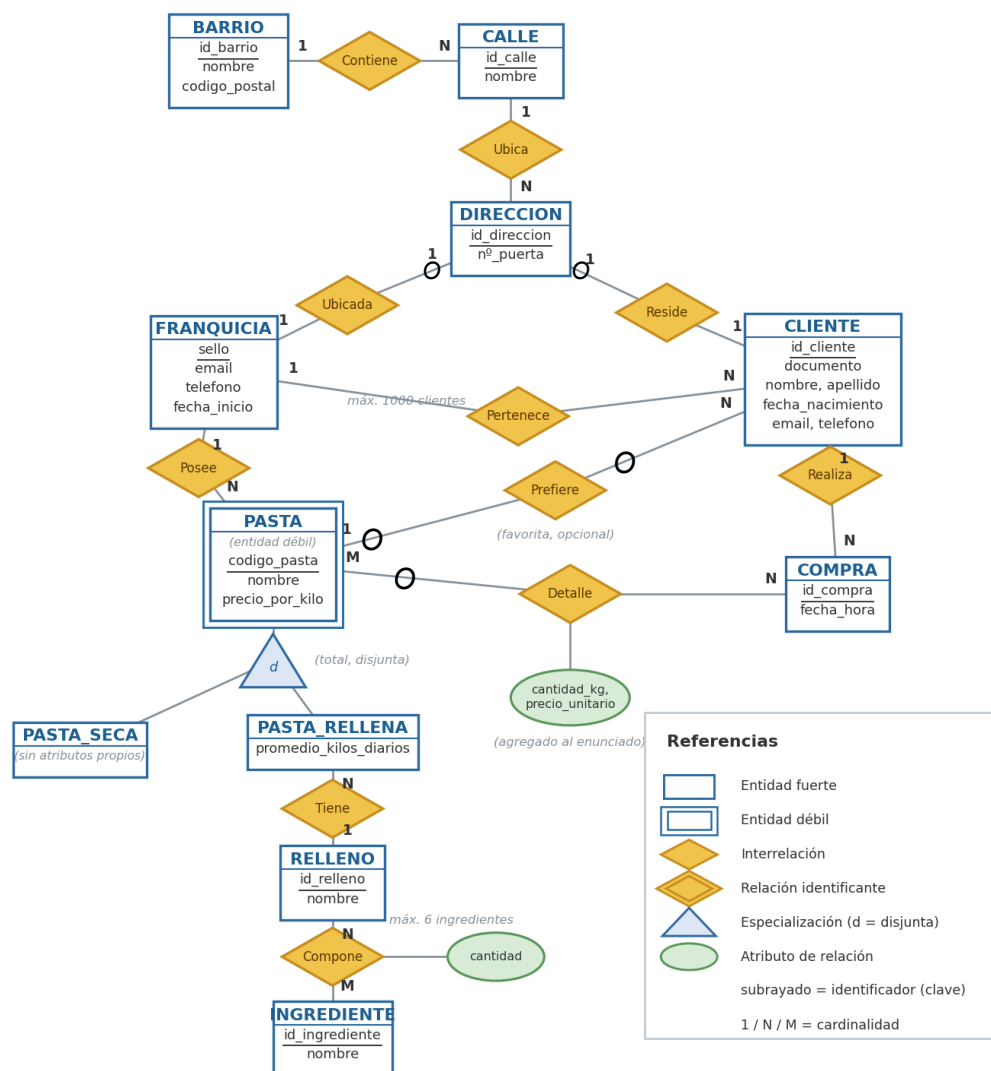


Diagrama Entidad-Relación del modelo (DER).

1.2 Implementación en PostgreSQL

El esquema está compuesto por trece tablas, cada una con sus tipos de datos, su clave primaria, sus claves foráneas y sus restricciones, y sin ningún índice. La siguiente tabla resume cómo se identifica cada entidad y a qué otras tablas hace referencia:

Tabla	Clave primaria (PK)	Claves foráneas (FK)
barrio	id_barrio	—
calle	id_calle	barrio
direccion	id_direccion	calle
franquicia	sello	direccion
pasta	(sello, codigo_pasta)	franquicia
pasta_seca	(sello, codigo_pasta)	pasta
pasta_rellena	(sello, codigo_pasta)	pasta, relleno
relleno	id_relleno	—
ingrediente	id_ingrediente	—
relleno_ingrediente	(id_relleno, id_ingrediente)	relleno, ingrediente
cliente	id_cliente	franquicia, direccion, pasta (favorita)
compra	id_compra	cliente
detalle_compra	(id_compra, sello, codigo_pasta)	compra, pasta

En cuanto a los tipos de datos, usamos SERIAL para los identificadores autogenerados, VARCHAR para las claves naturales (como el sello), NUMERIC para precios y kilos, y DATE o TIMESTAMP para las fechas. Las reglas de negocio descritas antes se implementan combinando restricciones CHECK, restricciones UNIQUE, claves foráneas y dos triggers: uno que limita la cantidad de ingredientes por relleno y otro que limita la cantidad de clientes por franquicia.

1.3 Poblado y Validación de Datos

Los datos se generan con la librería Faker (en local es_AR) y una semilla fija de valor 42, lo que hace que el dataset sea completamente reproducible. El generador respeta todas las restricciones del esquema y, además, busca generar información variada y realista. La siguiente tabla muestra cuántos registros quedaron cargados en cada tabla:

Tabla	Registros
detalle_compra	71.140 (principal)
compra	28.496 (principal)
cliente	7.977 (principal)
direccion	7.992
pasta	378
pasta_seca	204
pasta_rellena	174
relleno_ingrediente	209
calle	190

relleno	50
ingrediente	45
barrio	20
franquicia	15

Para asegurarnos de que los datos cargados son consistentes, el archivo 03_validacion.sql ejecuta dos tipos de verificaciones. La Parte A son chequeos de integridad pensados para que cada uno devuelva cero filas: buscan franquicias con más de mil clientes, rellenos con más de seis ingredientes, pastas sin subtipo o con ambos, favoritas de otra franquicia, detalles de otra franquicia, compras sin detalle, compras anteriores al inicio de la franquicia, direcciones huérfanas o compartidas, jerarquías geográficas incompletas y números de puerta inválidos. Si todos devuelven cero, sabemos que ninguna regla de negocio fue violada. La Parte B, en cambio, calcula estadísticos mediante funciones de agrupación: cuenta las filas de cada tabla y resume las columnas numéricas y categóricas.

SQL Avanzado

2.1 Funciones de Ventana

Buscamos plantear 2 preguntas de negocio que no sean trivialmente resolubles con un GROUP BY y encontrar una respuesta usando las funciones de ventana.

¿Cómo fue la evolución de las recaudaciones mensuales en las franquicias de Palermo en el último año?

Con esto buscamos ver si tenemos alguna franquicia que no logre aumentar ganancias y haya que prestarle atención.

La función de ventana *LAG()* es la opción óptima porque con una CTE previa podemos comparar, el mes actual con el anterior, directamente en la memoria RAM resolviendo el problema rápidamente. La alternativa en SQL tradicional nos obligaría a hacer un *SELF-JOIN* complejo manejando las fechas, lo que forzaría al motor a escanear la tabla en el disco duro múltiples veces, elevando el tiempo de ejecución.

Hay 3 franquicias en Palermo, veamos la “FR-004”:

	sello character varying (20)	mes_anio text	ingreso_mensual numeric	ingreso_mes_anterior numeric	crecimiento_neto numeric	porcentaje_crecimiento numeric
1	FR-004	2025-06	1947312.48030	2458186.96739	-510874.48709	-20.78
2	FR-004	2025-07	1935964.58640	1947312.48030	-11347.89390	-0.58
3	FR-004	2025-08	1916469.79561	1935964.58640	-19494.79079	-1.01
4	FR-004	2025-09	2757202.46713	1916469.79561	840732.67152	43.87
5	FR-004	2025-10	2112833.62351	2757202.46713	-644368.84362	-23.37
6	FR-004	2025-11	1850328.68999	2112833.62351	-262504.93352	-12.42
7	FR-004	2025-12	2038019.87410	1850328.68999	187691.18411	10.14
8	FR-004	2026-01	2146621.19164	2038019.87410	108601.31754	5.33
9	FR-004	2026-02	2201195.11472	2146621.19164	54573.92308	2.54
10	FR-004	2026-03	2355718.13009	2201195.11472	154523.01537	7.02
11	FR-004	2026-04	2257132.55951	2355718.13009	-98585.57058	-4.18
12	FR-004	2026-05	1958635.42803	2257132.55951	-298497.13148	-13.22

Notamos que no le fue bien en la última mitad del año 2025 a pesar de haber mejorado en septiembre respecto a los meses anteriores, que se mantuvo constante y aumentando la ganancias en casi toda la primera mitad de 2026 hasta Abril que empezó a caer otra vez.

¿Cuáles son los 3 tipos de pastas más vendidas y las 3 menos vendidas en cada una de las franquicias?

Esta pregunta nos permite tener un mejor control de stock y ver donde se puede mejorar.

Una función de ventana es la herramienta ideal para resolver este problema ya que si utilizáramos un *GROUP BY* tradicional podríamos saber el total vendido de cada pasta pero perderíamos la capacidad de aislar únicamente las 3 primeras y las 3 últimas de cada local. Utilizando la función *DENSE_RANK*, podemos generar un ranking independiente para cada sucursal de manera directa.

A esto se le suma todo lo dicho en la pregunta anterior sobre la complejidad computacional y de lectura de código.

	sello character varying (20)	nombre_pasta character varying (100)	total_kilos_vendidos numeric	top_mas_ventas bigint	top_menos_ventas bigint
1	FR-001	Agnolotti	1147.387	1	[null]
2	FR-001	Panzotti	1025.885	2	[null]
3	FR-001	Fusilli	786.095	3	[null]
4	FR-001	Tagliatelle	330.939	[null]	1
5	FR-001	Rigatoni	341.228	[null]	2
6	FR-001	Tirabuzón	352.840	[null]	3
7	FR-002	Spaghetti	1736.204	1	[null]
8	FR-002	Sorrentinos	1108.828	2	[null]
9	FR-002	Ñoquis	1104.000	3	[null]
10	FR-002	Ravioles	464.856	[null]	1
11	FR-002	Tortellini	490.171	[null]	2
12	FR-002	Farfalle	498.202	[null]	3

Acá podremos observar los dos tops de las franquicias FR-001 Y FR-002.

En FR-001 las pastas más vendidas son los Agnolotti, los panzotti, y los Fusilli; mientras que los menos vendidos son los Tagliatelle, los Rigatoni y los Tirabuzón, cada una con sus respectivas cantidades.

En FR-002 las pastas más vendidas son los Spaghetti, los Sorrentinos, y los Ñoquis; mientras que los menos vendidos fueron los Ravioles, los Tortellini, y los Farfalle.

2.2 Funciones estadísticas

En esta parte vamos a trabajar con la tabla *detalles_compra* a la que le vamos a realizar un análisis estadístico

Primero veremos la cantidad total de filas, cantidad y porcentaje de filas con valor no nulo, cantidad de valores diferentes.

	nombre_columna text	total_filas bigint	filas_no_nulas bigint	porcentaje_no_nulos numeric	valores_diferentes bigint
1	id_compra	71140	71140	100.00	28496
2	sello	71140	71140	100.00	15
3	precio_unitario	71140	71140	100.00	378
4	cantidad_kg	71140	71140	100.00	4751
5	codigo_pasta	71140	71140	100.00	31

Podemos observar que todas nuestras columnas están completas y que no tienen ningún valor Null. En la columna de 'id_compra' se deja ver que a pesar de estar registrando 71140 registros solo se cuenta con 28496 compras distintas, las cuales están repartidas en 15 franquicias distintas y que se vendieron un total de 31 pastas distintas.

Ahora con nuestras columnas numéricas veremos; el desvío estándar, mínimo, P05, primer cuartil, mediana, promedio, tercer cuartil, P95, máximo, cantidad y porcentaje de ceros, cantidad y porcentaje de valores negativos, cantidad de outliers.

	variable text	desvio_estandar numeric	minimo numeric	p05 double precision	q1 double precision	mediana double precision	promedio numeric	q3 double precision	p95 double precision
1	cantidad_kg	1.37	0.250	0.489	1.432	2.634	2.62	3.808	4.763
2	precio_unita...	2062.29	1506.41	1756.49	2643.08	3930.95	4368.08	5828.88	8272.43

	maximo numeric	cant_ceros bigint	pct_ceros numeric	cant_negativos bigint	pct_negativos numeric	total_filas bigint	cant_outliers bigint
	5.000	0	0.00	0	0.00	71140	0
	8938.26	0	0.00	0	0.00	71140	0

En la parte de la cantidad_kg observamos:

- Promedio = 2.62 y Mediana = 2.634, por lo que podríamos decir que hay un leve sesgo a la izquierda.
- Desvío estándar = 1.37, considerando que la media es 2.62 obtenemos un coeficiente de variación mayor al 50% lo que indica una fuerte dispersión en el volumen de compra
- Mínimo = 0.250 = ¼ kg y Máximo = 5kg
- Vemos que no tenemos datos negativos y que no hay outliers por lo que no tenemos sospechas de fallas en la carga de los datos o alguna compra anómala.

En la parte de precio_unitario vemos:

- Promedio = 4457.28 y Mediana = 3915.67. Aquí se observa que la mediana < promedio por lo que la distribución es sesgada a derecha, rescatando que seguramente hay una mayor cantidad de ventas de pastas con un precio menor al promedio pero unas pocas con el precio unitario alto hacen que aumente la media sobre la mediana.
- Desvío estándar = 2062.29 indica una alta dispersión en nuestros precios logrando una convivencia entre pastas económicas y pastas premium.
- Mínimo = 1513.13 y Máximo = 8997.98
- No se registran outliers por lo que no hay sospechas de errores de tipeo en los precios unitarios de las pastas.

Por último buscamos visibilizar la frecuencia, porcentaje y ranking de nuestras columnas categóricas. Analizamos sólo el resultado de sello:

	variable text	categoria text	frecuencia numeric	porcentaje numeric	rank bigint
12	sello	FR-011	6167	8.67	1
13	sello	FR-008	5805	8.16	2
14	sello	FR-006	5790	8.14	3
15	sello	FR-002	5731	8.06	4
16	sello	FR-010	5279	7.42	5
17	sello	FR-003	5260	7.39	6
18	sello	FR-013	5091	7.16	7
19	sello	FR-015	4535	6.37	8
20	sello	FR-009	4399	6.18	9
21	sello	FR-007	4358	6.13	10
22	sello	Otros	18725	26.32	11

La variable sello muestra un comportamiento concentrado a nivel de sucursales. El top 10 de franquicias con mayor volumen de operaciones a nivel ítem acumula el 73.68% de la actividad dejando solo un 26.32% al resto de locales.

2.3 Análisis de Performance

Para esto vamos a tomar la consulta realizada en el ejercicio 2.1 pregunta 1:

“¿Cómo evolucionó la recaudación de las franquicias del barrio Palermo mes a mes en el último año?”

Plan de ejecución actual:

	QUERY PLAN text
1	Subquery Scan on mam (cost=1269.97..1400.26 rows=1922 width=167) (actual time=10.658..11.976 rows=36 loops=1)
11	-> Hash Join (cost=61.40..709.63 rows=774 width=12) (actual time=1.287..3.661 rows=2603 loops=1)
12	Hash Cond: (co.id_cliente = cl.id_cliente)
13	-> Seq Scan on compra co (cost=0.00..582.44 rows=15478 width=16) (actual time=0.007..1.997 rows=15474 loops=1)
44	Planning Time: 0.847 ms
45	Execution Time: 12.065 ms

Al analizar el plan inicial, observamos un costo total estimado de 1400.26 y un tiempo de ejecución de 12.065 ms. Notemos que al realizar el JOIN entre compra y cliente mediante el id_cliente se usa un Seq Scan, veamos que pasa si creamos el índice id_cliente en la tabla Compra.

Plan después de crear el índice:

	QUERY PLAN text
1	Subquery Scan on mam (cost=816.18..946.47 rows=1922 width=167) (actual time=9.638..10.940 rows=36 loops=1)
36	-> Index Scan using idx_compra_id_cliente on compra co (cost=0.29..0.48 rows=2 width=16) (actual time=0.001..0.001 rows=2 loops=1325)
37	Index Cond: (id_cliente = cl.id_cliente)
42	Planning Time: 0.984 ms
43	Execution Time: 11.005 ms

El cambio generó una modificación en la estrategia del optimizador del motor el cual optó por un Index Scan a la hora de realizar la junta con la tabla cliente. Podemos ver que el

costo total estimado de la consulta pasó a ser 946.47 y el tiempo de ejecución real se estabilizó en el orden de los 11ms

Procesamiento con Spark

3.1 MapReduce con Spark RDDs

Para esta resolución utilizamos principalmente los archivos `pasta.csv`, `detalle_compra.csv`, `compra.csv` y `cliente.csv`, cargados como RDDs de diccionarios. A partir de ellos implementamos cuatro procesamientos de negocio que permiten analizar el comportamiento comercial de las franquicias y de los productos.

Procesamiento 1: ¿Qué pasta fue comprada en la mayor cantidad de franquicias?

Acá buscamos ver qué pasta llegó a más lugares. No miramos tanto la cantidad total que se vendió, sino en cuántos locales diferentes se ofreció al menos una vez.

- **Fase de Map:**

Para resolver esta pregunta tomamos por un lado la información de `pasta.csv`, que nos permite vincular cada combinación de código de pasta y franquicia con su nombre comercial, y por otro lado `detalle_compra.csv`, que contiene las ventas registradas. En la primera transformación mapeamos `pasta.csv` a la estructura `((codigo_pasta, sello), nombre_pasta)`, generando un RDD auxiliar para recuperar el nombre del producto. Luego, desde `detalle_compra.csv`, cada fila se transforma en `((codigo_pasta, sello), 1)`. Esta clave representa que una pasta determinada fue comprada en una franquicia determinada. Como una misma pasta puede aparecer muchas veces en el mismo local, aplicamos `distinct()` para conservar una sola ocurrencia por combinación pasta-franquicia. Después de eso se realiza un `join()` con el RDD de nombres y se remapea el resultado para dejar pares de la forma `(nombre_pasta, 1)`.

- **Fase de Reduce:**

La fase de Reduce se implementa mediante `reduceByKey()`, sumando los unos asociados a cada `nombre_pasta`. De esta manera obtenemos, para cada pasta, la cantidad de franquicias distintas en las que fue comprada. Luego se calcula el máximo valor alcanzado usando la acción `max()`.

- **Resultado:**

El resultado final se imprime como una lista de tuplas del tipo `(nombre_pasta, cantidad_de_franquicias)`. En caso de empate, se devuelven todas las pastas que alcanzaron la misma cobertura máxima. Este procesamiento permite detectar cuales son los productos con mayor presencia transversal dentro de la cadena, algo útil para decisiones de catálogo, abastecimiento o promociones comunes a toda la red.

```
Pasta comprada en la mayor cantidad de franquicias:  
('Panzotti', 29)
```

Procesamiento 2: ¿Cuál es el gasto promedio por compra en cada franquicia?

Esta pregunta busca comparar el comportamiento de consumo entre locales, observando cuánto dinero gastan los clientes, en promedio, cada vez que realizan una compra en una franquicia específica.

- **Fase de Map:**

Para responder esta consulta partimos de detalle_compra.csv, donde cada compra puede estar compuesta por varios ítems. Calculamos el total de cada compra, primero multiplicamos la cantidad de kilos por el precio unitario y luego sumamos esos subtotales para obtener el total. El primer map transforma detalle_compra.csv en pares (id_compra, subtotal). En paralelo, desde compra.csv se genera el RDD (id_compra, id_cliente) y desde cliente.csv el RDD (id_cliente, sello). Luego de enlazar estos RDDs con join(), los registros se remapean para llegar a una estructura final de la forma (sello, (monto_compra, 1)), donde el primer elemento representa el total monetario de una compra y el segundo sirve como contador.

- **Fase de Reduce:**

La reducción se realiza con reduceByKey(), sumando para cada franquicia tanto los montos acumulados como la cantidad total de compras con lo que se calcula el promedio. Así, cada clave sello queda asociada a un par (monto_total, prom_gasto).

- **Resultado:**

El resultado final se devuelve ordenado de menor a mayor como una lista de tuplas (sello, promedio), utilizando takeOrdered(). Este análisis permite comparar el valor promedio del ticket entre franquicias y puede servir para estudiar diferencias de demanda, poder adquisitivo de la clientela o desempeño comercial de cada local.

```
Promedio de gasto por compra en cada franquicia:
('FR-012', 25765.06)
('FR-005', 25862.58)
('FR-013', 26027.63)
('FR-015', 26177.24)
('FR-003', 26304.69)
('FR-002', 26659.22)
('FR-014', 28283.13)
('FR-001', 28596.18)
('FR-010', 28948.29)
('FR-006', 29226.78)
('FR-007', 30025.35)
('FR-008', 30044.1)
('FR-011', 31182.03)
('FR-009', 32043.34)
('FR-004', 33392.35)
```

Procesamiento 3: ¿Cuáles son las 10 pastas más vendidas por kilos?

En este caso buscamos identificar qué productos concentran el mayor volumen físico de ventas, independientemente de la franquicia en la que fueron comercializados. Esta consulta permite detectar cuales son las pastas con mayor salida real en términos de cantidad, más allá del valor monetario de cada operación.

- **Fase de Map:**

Con la información de detalle_compra.csv, agrupamos primero por combinación (sello, codigo_pasta) para obtener el total vendido por local, y luego unimos esa información con pasta.csv para recuperar el nombre de cada producto. Cada fila de detalle_compra.csv se transforma en ((sello, codigo_pasta), cantidad_kg). En paralelo, desde pasta.csv se genera el RDD ((sello, codigo_pasta), nombre_pasta), que luego se usa para recuperar el nombre comercial del producto mediante un join(). Después del join, se remapea el resultado como (nombre_pasta, cantidad_kg).

- **Fase de Reduce:**

La primera reducción se realiza sobre la clave (sello, codigo_pasta) para sumar todos los kilos vendidos de una misma pasta dentro de una franquicia. La segunda reducción se hace sobre la clave nombre_pasta, sumando los kilos provenientes de distintos sellos para obtener el total vendido en toda la red. Finalmente se ordena en

forma descendente por cantidad de kilos y se limita el resultado a los diez primeros elementos.

- **Resultado:**

El resultado final se imprime como un ranking de diez tuplas del tipo (nombre_pasta, cantidad_kg), ordenadas de mayor a menor. Este procesamiento permite identificar los productos con mayor rotación física, lo que puede ser útil tanto para la planificación de producción como para decisiones de stock y abastecimiento de materias primas.

```
Top 10 pastas por kilos vendidos:
('Panzotti', 14392.937)
('Capeletti', 12756.074)
('Ravioloni', 11157.025999999998)
('Ravioles', 10305.064000000002)
('Fusilli', 9764.783)
('Ñoquis', 8372.837)
('Penne', 8349.024000000001)
('Tortellini', 8101.1630000000005)
('Mezzelune', 7940.683)
('Cintas', 7851.788000000001)
```

Procesamiento 4: ¿Qué meses concentran la mayor cantidad de compras?

Esta consulta busca detectar patrones estacionales en la demanda, identificando qué meses reúnen la mayor cantidad de tickets emitidos. Al tratarse de un conteo agregado independientemente del año, el análisis permite observar si existe una concentración recurrente de compras en ciertos momentos del calendario.

- **Fase de Map:** A partir de compra.csv, se extrae el mes (MM) del campo fecha_hora. Cada registro se transforma y emite como la tupla (mes, 1), donde la clave es el mes calendario y el valor 1 representa una ocurrencia de compra.
- **Fase de Reduce:** Se utiliza reduceByKey() para sumar todos los unos asociados a la misma clave, consolidando el total en (mes, cantidad_compras). Finalmente, se aplica sortBy() para ordenar la colección de forma descendente según la cantidad.
- **Resultado:** El resultado final se imprime como una lista de tuplas (mes, cantidad_compras), ordenadas desde el mes con más compras hasta el de menor actividad. Este análisis permite detectar meses de alta y baja demanda.

```
Meses con mayor cantidad de compras:
('05', 2446)
('12', 2444)
('01', 2430)
('11', 2410)
('03', 2390)
('06', 2389)
('04', 2386)
('08', 2378)
('07', 2367)
('09', 2341)
('10', 2301)
('02', 2214)
```

Transformaciones vs Acciones:

Un aspecto importante de esta etapa es distinguir entre transformaciones y acciones dentro del modelo de ejecución de Spark. Las operaciones como map(), filter(), distinct(), join(), reduceByKey() y sortBy() son transformaciones: construyen un nuevo RDD a partir de otro,

pero no ejecutan inmediatamente el cómputo. Spark las registra de manera diferida, armando el linaje de operaciones que deberá aplicar más adelante.

El procesamiento real ocurre recién cuando se invoca una acción, es decir, una operación que necesita materializar un resultado concreto. En nuestro notebook, esto sucede con acciones como `collect()`, `max()` y `takeOrdered()`. Recién en ese momento Spark evalúa todas las transformaciones previas, distribuye las tareas entre las particiones y ejecuta el pipeline completo necesario para producir la salida pedida.

Esta evaluación lazy resulta beneficiosa porque Spark puede optimizar el plan de ejecución, evitar cálculos innecesarios y coordinar mejor el trabajo distribuido. Por eso, aunque en el notebook se encadenan múltiples transformaciones, el cómputo no se dispara hasta que se solicita explícitamente un resultado final para mostrar o recuperar.

Persistencia Políglota — NoSQL

Redis

Redis es un almacén de estructuras de datos en memoria, y lo incorporamos para resolver aquello que conviene mantener fuera de PostgreSQL: lecturas individuales por ID muy frecuentes, colas de trabajo y datos que deben expirar por sí solos.

4.1.1 Modelo Clave-Valor y Hashes

Identificamos tres datos del dominio que se consultan permanentemente de a uno por su identificador, por lo que resultan naturales de representar como pares clave-valor:

Perfil de cliente → HASH, bajo la clave cliente:{id_cliente}.

Ficha de pasta → HASH, bajo la clave pasta:{sello}:{codigo_pasta}.

Precio por kilo → STRING, bajo la clave precio:{sello}:{codigo_pasta}.

Elegimos hashes para los objetos compuestos por varios campos (como el cliente y la pasta) y un string simple para el precio, que es un valor único, se lee en cada venta y cambia con frecuencia. La carga se realiza con `hset` para los hashes y `set` para el string; las consultas, con `hgetall` para obtener el perfil completo del cliente, `hget` para un campo puntual de una pasta y `get` para el precio. Para demostrar la actualización, modificamos el precio de una pasta con `hset`, lo replicamos en el string con `set` y mostramos el valor antes y después del cambio.

Resolvemos esto en Redis y no en PostgreSQL porque estas fichas se consultan en cada pantalla y en cada venta: atender cada lectura con un `SELECT` o un `JOIN` sobre la base relacional resulta costoso. Redis devuelve el objeto completo con un `HGETALL` en tiempo $O(1)$ sobre memoria, funcionando como una caché de lectura que descarga a la base relacional.

4.1.2 Listas como Colas o Historial

Para las listas planteamos una cola de producción FIFO por franquicia, identificada con la clave cola_produccion:{sello}: cada franquicia mantiene su propia cola con los pedidos pendientes de fabricar. Los pedidos nuevos ingresan por el final con `Rpush` y la cocina toma el siguiente desde el inicio con `Lpop`, de modo que se respeta el orden de llegada.

Para poblar la cola tomamos pedidos reales de detalle_compra (franquicia FR-001) y los insertamos como JSON con rpush. Luego la inspeccionamos con llen, para saber cuántos pedidos hay, y con lrange, para verla completa. Por último simulamos el flujo de trabajo: ingresa un pedido nuevo con rpush y la cocina produce los siguientes con lpop, mostrando cómo la cola se va vaciando.

Una lista de Redis es la estructura adecuada en este caso porque ofrece semántica de cola con operaciones $O(1)$ en ambos extremos, sin necesidad de una tabla de estado, de sucesivos UPDATE ni del manejo de concurrencia que requeriría PostgreSQL. Además, el pedido es efímero: desaparece de la cola en cuanto se produce.

4.1.3 TTL: Datos con Tiempo de Vida

El último uso aprovecha el vencimiento automático de claves. Definimos tres datos que tiene sentido que expiren por sí solos, con tiempos distintos según su propósito:

Reserva de un pedido en armado (reserva:{sello}:{id_compra}), con un TTL de 600 segundos (10 minutos): mientras el cliente arma el pedido se le reservan los kilos, y si no confirma a tiempo la reserva se libera sola para no comprometer la producción de forma indefinida.

Promoción por tiempo limitado (promo:{sello}:{codigo_pasta}), con un TTL de 3600 segundos (1 hora): una oferta acotada que debe desaparecer por sí misma al terminar la ventana comercial.

Caché de una consulta costosa (el ranking de pastas más vendidas) (cache:top_pastas:{sello}), con un TTL de 300 segundos (5 minutos): tolera estar algo desactualizada, por lo que se recalcula cada tanto en lugar de consultar PostgreSQL en cada visita.

Las tres claves se crean indicando su vencimiento con set ... ex=segundos y consultamos el tiempo restante con ttl. Para ilustrar el comportamiento mostramos mensajes del estilo “la reserva sigue vigente por N segundos” y, para demostrar la expiración sin tener que esperar, creamos una reserva con un TTL de 2 segundos: tras la espera, ttl devuelve -2, lo que confirma que la clave ya no existe.

Conviene remarcar que el TTL es una decisión de diseño y no sólo un detalle técnico: una reserva que nunca expira dejaría kilos comprometidos para siempre y bloquearía la producción; un TTL demasiado corto en la caché anularía su beneficio; y el de la promoción debe coincidir con la ventana comercial para no vender a un precio que ya venció. Como ventaja adicional, el dato se elimina solo, sin necesidad de un proceso de limpieza.

Esta sección se conecta, además, con la cola de producción anterior: la reserva es el paso previo a la cola. El pedido reserva sus kilos mientras el cliente confirma y, una vez confirmado, ingresa a cola_produccion:{sello} para su fabricación.

4.2 MongoDB

Para la resolución de esta etapa, se optó por la alternativa de despliegue local utilizando Docker.

Pasos de configuración implementados:

- Imagen y Servicio: se utilizó la imagen oficial mongo:7.

- Seguridad y Autenticación: se decidió implementar autenticación estricta mediante las variables de entorno MONGO_INITDB_ROOT_USERNAME y MONGO_INITDB_ROOT_PASSWORD para emular un entorno productivo real.
- Mapeo de Puertos: Se expuso el puerto 27017:27017 para la comunicación desde el cliente PyMongo.
- Persistencia de Datos: Se configuró un volumen nombrado (mongodata_secure:/data/db) para garantizar que la información de las colecciones persista entre distintos reinicios del contenedor.
- Para establecer la conexión desde el notebook hacia el contenedor se utilizó el siguiente string de conexión:

"mongodb://<usuario>:<password>@localhost:27017/"

Durante la configuración del entorno, se presentó un problema debido a que al principio creamos el contenedor para que se levante inicialmente sin user y password, al crearlas y reiniciar el contenedor se ignoraba las nuevas credenciales aplicadas en el archivo Compose. Para esto creación de una instancia limpia modificando la referencia del volumen (de mongodata a mongodata_secure), lo que obligó al motor a ejecutar la inicialización de seguridad correctamente.

4.2.1 Diseño de Colecciones

Para este paso decidimos armar una colección de documentos para representar las pastas y otra colección para modelar los clientes. A continuación dejamos como son las estructuras completas de cada colección y algunas aclaraciones en negrita

```

Cliente = {
  "_id": <id_objeto>,
  "sello": <id_franquicia>, (Referencia por ID - compuesta con cod_pasta_favorita)
  "nombre": <nombre_cliente>,
  "apellido": <apellido_cliente>,
  "documento":
  "email": <email>
  "telefono": <telefono>, (Opcional)
  "fecha_nacimiento": <fecha_nacimiento>, (Opcional)
  "direccion": {
    "calle": <nombre_calle>,
    "numero": <numero_casa>,
    "barrio": <nombre_barrio>,
    "cod_postal": <cod_postal>
  }, (Direccion embebida)
  "cod_pasta_favorita": <cod_pasta> (Referencia por ID junto a sello - Opcional)
}

```

```

Pasta = {
  "_id": <id_objeto>,
  "sello": <id_franquicia>, (Referencia por ID a la franquicia - acá no lo usamos)
  "cod_pasta": <cod_pasta>,
}

```

```

"nombre": <nombre_pasta>,
"precio_por_kg": <precio_por_kg>,
"tipo": <S/R>,
"relleno":{
    "nombre": <nombre_relleno>,
    "ingredientes": [ {"nombre": <nombre_ing>,
                        "cantidad": <cantidad_ing>},...
    ] ( Longitud del array puede variar de 1 a 6)
} (Datos embebidos del relleno solo están si "tipo" = R)

```

En nuestra colección de documentos Clientes, tenemos referencia por ID a pastas con (Sello,cod_pasta_favorita), a su vez, sello también funciona como referencia por ID a Franquicia. Por otro lado, decidimos embeber toda la información relacionada con la dirección, al desnormalizar la estructura de la etapa 1 ya no es necesario realizar múltiples juntas para recopilar todos los datos geográficos del cliente.

La colección de Pastas tiene referencia por ID a las franquicias con el sello, esto nos permite vincular a cada cliente con la sucursal a la que compra. En este modelo aprovechamos la flexibilidad de NoSQL para implementar un esquema polimórfico a diferencia del modelo planteado en SQL, logrando que las pastas secas (S) y las rellenas (R) vivan en la misma colección. Esto permite obtener todos los ingredientes de la pasta rellena y sus cantidades mucho más rápido ya que estamos condensando 5 tablas SQL en un solo documento.

Restricciones de dominio y reglas de negocio que se tomaron en cuenta a la hora de inyectar los datos:

- Identificadores: Los clientes se identifican unívocamente por su DNI y las pastas usan una clave compuesta (código de pasta, sello de franquicia).
- Pastas secas: El código de producto es único y determina el nombre, siendo el mismo en todas las franquicias.
- Pastas rellenas: Los ingredientes y cantidades están predefinidos por el nombre del relleno. El código de producto está determinado por la combinación exacta de tipo de pasta + relleno.
- Direcciones: Limitadas a CABA, respetando la correspondencia real entre el barrio y su código postal.
- Precios: Secas entre \$2500 y \$4500; rellenas entre \$5000 y \$8500.

4.2.3 Aggregation Pipelines

Ranking de Pastas más elegidas

“¿Cuáles son los tipos de pasta que más prefieren nuestros clientes globalmente, independientemente de la franquicia donde compran o el relleno que elijan? “

Etapas y sus transformaciones:

- Etapa 1 - Filtrado: Tomamos la colección clientes y mediante “\$match” y “\$exists” nos quedamos solo con los que tengan una pasta favorita.

- Etapa 2 - Junta: Aplicamos "\$lookup" para realizar una junta entre los clientes con pasta favorita y las pastas a través de los campos "cod_pasta_favorita" y "cod_pasta".
- Etapa 3 - Agrupación: Con todos los datos que necesitamos procedemos a agrupar todos los documentos con mismo nombre de pasta, agregando un contador para saber cuántos clientes la tienen como favorita.
- Etapa 4 - Ordenamiento y función de ventana: Ordenamos la salida por su contador y junto a una función de ventana le asignamos una posición según el favoritismo.
- Etapa 5 - Proyección: Elegimos los campos de interés para mostrar que en nuestro caso son ; "ranking", "nombre", "código", "tipo", "cantidad".

Ranking Comercial de Rellenos

"¿cuál es el ranking de popularidad de nuestros rellenos?"

Etapas y sus transformaciones:

- Etapa 1 - Filtrado: Al igual que en la primera etapa del pipeline anterior comenzamos filtrando a los cliente con pasta favorita.
- Etapa 2 - Junta: Volvemos a realizar la junta entre la colección pastas y clientes mediante los campos "cod_pasta_favorita" y "cod_pasta".
- Etapa 3 - Filtrado: Luego de aparear los datos de los clientes con los de su pasta favorita buscamos quedarnos solo con aquellos que tengan una pasta rellena usando el "\$match".
- Etapa 4 - Agrupación: Pasamos a juntar nuestros documentos que tengan el mismo nombre de relleno mediante el "\$group" y sumamos cuantas veces aparece cada uno.
- Etapa 5 - Ordenamiento: Ordenamos la salida por sus apariciones y junto a una función de ventana le agregamos un ranking.
- Etapa 6 - Proyección: Elegimos los campos de interés para mostrar que en nuestro caso son ; "nombre", "ranking", "ingredientes.nombre", "cantidad".